

Containerized Infrastructure and CI/CD Automation using Docker and Jenkins

Project: NUFindIt (Lost-and-Found Board)

Course: BSIT ITE 301, System Architecture and Integration

Group name: NUFindIt Team

Repository: <https://github.com/kaaaaaailll/nufindit-devops>

1. Team Members and Roles

Member	Role
Salenillas, Mikhail John Alexis N.	DevOps / CI-CD Engineer
Aboy, Jean Ice Vincent D.	Infrastructure Engineer
Dizon, Kizen Jared D.	Project Lead
Mallari, Ian Rose C.	Backend and Database Engineer
Flores, Nicole James B.	Frontend, QA, and Documentation Lead

2. Project Description and Modules

NUFindIt is a small lost-and-found board where users register, log in, and post found or lost items. The application itself is deliberately simple. The focus of the project is the infrastructure: every module runs in its own Docker container, and Jenkins builds, tests, and deploys the whole system automatically on every push to GitHub.

Module	Purpose	Port (internal)
proxy	Nginx reverse proxy. The only service published to the host (8080). Routes / to the frontend, /api/items/ to items-api, /api/auth/ to auth-api.	80
frontend	Static HTML page served by Nginx. The footer shows the build number ("Build: N"), which proves which version is live.	80
items-api	Node.js 20 + Express REST API. Endpoints: GET /health, GET /items, POST /items. Stores data in PostgreSQL. Covered by Jest tests.	3000
auth-api	Node.js 20 + Express API using bcryptjs and jsonwebtoken. Endpoints: GET /health, POST /register, POST /login. Covered by 5 Jest tests.	3001
db	PostgreSQL 16 (alpine), built from ./db. Data is kept in the named volume db-data.	5432
jenkins	Separate Compose stack in infra/jenkins/. Runs the CI/CD pipeline. UI at http://localhost:8081 .	8080 (mapped to 8081)

All application modules share one user-defined Docker network (nufindit-net). Only the proxy is reachable from outside; the APIs and the database are not exposed to the host.

3. Architecture

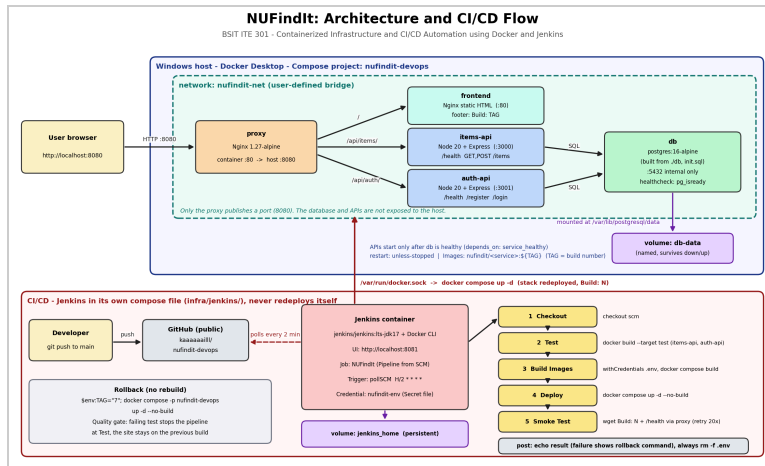


Figure 1. NUFindIt system architecture.

How a request flows. The browser talks only to the Nginx proxy on port 8080. The proxy decides where to send the request by its URL path:

- `/` goes to the frontend container, which serves the static page.
- `/api/items/` goes to items-api (port 3000).
- `/api/auth/` goes to auth-api (port 3001).

Both APIs read and write data in the PostgreSQL container (db) over the private network `nufindit-net`. Nothing except the proxy publishes a port to the host, so the APIs and the database cannot be reached directly from outside.

Why this design.

- **One entry point.** The proxy hides the internal services, gives the browser a single origin (no CORS problems), and makes it easy to add or replace modules.
- **One container per module.** Each module has its own image, so it can be built, tested, restarted, and rolled back independently.
- **Data outside the container.** The database files live in the named volume `db-data`, not in the container. Containers can be destroyed and recreated on every deploy while the data stays.
- **Startup order.** `depends_on` with `condition: service_healthy` makes the APIs wait until PostgreSQL passes its health check.

CI/CD flow. A developer pushes to the `main` branch on GitHub. Jenkins (running in its own container) detects the new commit by polling, runs the tests, builds the images tagged with the build number, redeploys the Compose stack, and finishes with a smoke test through the proxy. Details are in sections 7 and 8.

4. Technology Stack

Layer	Technology	Justification
Containers	Docker (Docker Desktop on Windows)	Required by the project. Packages each module with its dependencies so it runs the same on every team member's laptop and in Jenkins.
Orchestration	Docker Compose v2	Describes all five services, the network, the volume, health checks, and start order in one file. Simple and enough for a single-host deployment.
Reverse proxy	Nginx 1.27-alpine	Lightweight, widely used, and simple path-based routing. Alpine keeps the image small. Pinned version, not <code>latest</code> .
Frontend	Static HTML served by Nginx 1.27-alpine	The app is not the grading focus, so a static page keeps the module small. It also lets the build number be injected at build time and shown in the footer.
Backend runtime	Node.js 20 (alpine) + Express	Fast to develop, small images, and the team already knows JavaScript. Node 20 is an LTS release.

Layer	Technology	Justification
Database driver	pg	The standard PostgreSQL client for Node.
Authentication	bcryptjs + jsonwebtoken	bcryptjs hashes passwords (pure JavaScript, no native build step in alpine). JWT gives stateless login tokens.
Database	PostgreSQL 16-alpine	Reliable relational database with an official image and a built-in <code>pg_isready</code> check used for the health check.
Testing	Jest	Simple test runner for Node. Runs inside the Docker <code>test</code> stage, so the same tests run on every laptop and in Jenkins.
CI/CD server	Jenkins LTS (jenkins/jenkins:lts-jdk17)	Required by the project. Declarative pipelines stored in the repository (Jenkinsfile).
Pipeline trigger	Poll SCM, <code>H/2 * * * *</code>	Jenkins runs on a laptop without a public address, so GitHub cannot call a webhook. Polling every 2 minutes needs no extra tool.
Source control	Git + GitHub (public repository)	Shared history for the whole team, and the instructor can access it.
Secrets	Jenkins "Secret file" credential (<code>nufindit-env</code>) + git-ignored <code>.env</code>	Passwords and the JWT secret never enter the repository. Only <code>.env.example</code> is committed.
Developer tools	VS Code with PowerShell terminal	What the team used day to day.

5. Dockerfile Explanations

Every module we wrote has its own Dockerfile and its own `.dockerignore` (items-api, auth-api, frontend, proxy, db). Only the Jenkins image (infra/jenkins) is covered in Section 7.

5.1 Summary

Module	Base image	Multi-stage	Non-root user	HEALTHCHECK
items-api	node:20-alpine	Yes (base, test, production)	Yes (USER node)	Yes (port 3000)
auth-api	node:20-alpine	Yes (base, test, production)	Yes (USER node)	Yes (port 3001)
frontend	nginx:1.27-alpine	No	No (default nginx user setup)	No (checked by the smoke test)
proxy	nginx:1.27-alpine	No	No (default nginx user setup)	No (checked by the smoke test)
db	postgres:16-alpine	No	No (image default)	In docker-compose.yml (Section 6)

5.2 items-api and auth-api (multi-stage)

The two APIs use the same structure. Only the port differs (3000 and 3001).

- **base stage:** starts from `node:20-alpine`, sets `WORKDIR /app`, copies `package*.json` first and runs `npm ci`, then copies the rest of the code. Copying the dependency files first lets Docker reuse the cached `npm ci` layer when only source code changes, so builds are faster.
- **test stage:** FROM base, then `RUN npm test`. Jenkins runs `docker build --target test`. If any Jest test fails, the build fails and the pipeline stops before Deploy. Because the tests run inside Docker, the Jenkins machine needs no Node.js installed.
- **production stage:** a fresh `node:20-alpine` image. It sets `NODE_ENV=production`, runs `npm ci --omit=dev` (no test tools), and copies only the `src` folder from the base stage. The image therefore contains no tests and no dev dependencies, which makes it smaller.
- **Security:** USER node runs the app as the non-root user that the Node image provides.
- **HEALTHCHECK:** every 15 seconds the container runs `wget` against `/health` on its own port. Docker marks the container healthy or unhealthy, and `docker compose ps` shows it.
- **CMD:** `node src/server.js` starts the service.

5.3 frontend

FROM nginx:1.27-alpine with a build argument BUILD_NUMBER (default dev). The Dockerfile copies index.html into the nginx web root, then a sed command replaces the placeholder **BUILD** with the build number. This is how the footer shows "Build: ". docker-compose.yml passes TAG as the build argument, so the footer proves which version is live.

5.4 proxy

FROM nginx:1.27-alpine, then nginx.conf is copied to /etc/nginx/conf.d/default.conf. The configuration is baked into the image, not bind-mounted, so it works the same when Jenkins deploys. The proxy routes / to the frontend, /api/items/ to items-api and /api/auth/ to auth-api.

5.5 db

FROM postgres:16-alpine, then init.sql is copied to /docker-entrypoint-initdb.d/. The official image runs that script once, on first start, when the data directory is empty. Because the script is copied into the image, no bind mount of ./db/init.sql is needed. This fixed a problem that appeared when Jenkins deployed from its own workspace (Section 11). The data itself lives in the named volume db-data, so it survives redeployments.

5.6 Choices and limitations

- **Pinned versions:** node:20-alpine, nginx:1.27-alpine and postgres:16-alpine are pinned to a major or minor version. None uses :latest. A major-version tag can still receive security patches, so it is not a byte-exact pin. A digest would be exact.
- **Alpine:** small images with fewer packages, so a smaller attack surface.
- **.dockerignore:** each module excludes files that should not enter the build context (for example node_modules and local files), which keeps builds fast and keeps local files out of the image.
- **Limitations:** the frontend, proxy and db images use the default users of their official images and have no HEALTHCHECK in the Dockerfile. The APIs, which hold our own code, have both. Possible improvements: a non-root nginx image and a Dockerfile HEALTHCHECK for the proxy.

6. Docker Compose Explanation

One file, docker-compose.yml, describes the whole application stack. One command starts everything: `docker compose up -d --build`. Jenkins uses the same file with `docker compose up -d --no-build`. Jenkins itself is in a separate compose file (Section 7).

6.1 Services

Service	Image name	Built from	Published port	Depends on
proxy	nufindit/proxy:\${TAG:-dev}	./proxy	8080 to 80	frontend, items-api, auth-api (start order only)
frontend	nufindit/frontend:\${TAG:-dev}	./frontend (build arg BUILD_NUMBER)	none	none
items-api	nufindit/items-api:\${TAG:-dev}	./items-api, target production	none	db (service_healthy)
auth-api	nufindit/auth-api:\${TAG:-dev}	./auth-api, target production	none	db (service_healthy)
db	built from ./db, no explicit image tag	./db	none	none

Only the proxy publishes a port (8080 on the host). The APIs, the frontend and the database are reachable only inside the Docker network, so the database is never exposed to the host or the public.

6.2 Image tags and rollback

Every application image is named nufindit/\${TAG:-dev}. Jenkins sets TAG to the build number, so build 22 produces nufindit/items-api:22 and so on. Without TAG the default is dev. Old tags stay on the machine, so a rollback is one command:

```
$env:TAG="18"; docker compose -p nufindit-devops up -d --no-build
```

The frontend receives TAG as the build argument BUILD_NUMBER, so the page footer shows "Build: ".
Note: the db service has no image: line, so its image is not tagged by build number. A rollback changes the application services, not the db image. The data stays in the db-data volume.

6.3 Startup order and health checks

- items-api and auth-api use `depends_on` with `condition: service_healthy`. They start only after the db health check passes.
- The db health check runs `pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}` every 10 seconds, with a 5 second timeout and 10 retries. The double `$$` stops Compose from substituting the variable, so the container's own environment value is used.
- The two APIs also have a HEALTHCHECK inside their Dockerfiles (Section 5). `docker compose ps` shows db, items-api and auth-api as healthy.
- The proxy lists frontend, items-api and auth-api in a plain `depends_on` list. That only controls the start order. It does not wait for those services to be healthy.

6.4 Network

All five services are attached to one user-defined bridge network, `nufindit-net`. Compose adds the project name, so it appears in Docker as `nufindit-devops_nufindit-net`. Containers reach each other by service name (for example the API connects to host db, and the proxy forwards to items-api). Jenkins is on its own network, `jenkins_default`.

6.5 Volume and persistence

The named volume `db-data` is mounted at `/var/lib/postgresql/data` in the db container. Docker shows it as `nufindit-devops_db-data`. The volume is separate from the container, so records survive `docker compose down`, redeployments by Jenkins and rollbacks. Only `docker compose down -v` would delete it, so we never use `-v`.

6.6 Environment variables and secrets

Variable	Used by	Purpose
DB_NAME	db, items-api, auth-api	Database name
DB_USER	db, items-api, auth-api	Database user
DB_PASSWORD	db, items-api, auth-api	Database password
JWT_SECRET	auth-api	Signing key for login tokens
TAG	all built images, frontend	Image tag and build number shown in the footer

DB_HOST is fixed to db in the compose file because that is the service name on the network. The real values are in `.env`, which is listed in `.gitignore` and is never committed. The repository contains `.env.example` with placeholder values. In the pipeline, Jenkins copies the Secret file credential `nufindit-env` to `.env` for the run and deletes it afterwards.

6.7 Restart policy and project name

Every service has `restart: unless-stopped`, so the stack comes back after Docker Desktop or the laptop restarts. We saw this happen: the containers started again on their own. No service uses `container_name`, and we always pass the project name with `-p nufindit-devops`, so every command manages the same stack.

7. Jenkins Setup

7.1 How Jenkins was installed

Jenkins runs in its own container, defined in a separate compose file (`infra/jenkins/docker-compose.yml`). It is kept apart from the application stack so the pipeline never redeployes Jenkins itself. The two stacks use different networks (`jenkins_default` and `nufindit-net`) and different project names.

Start command: `cd infra/jenkins` then `docker compose up -d --build`. The web UI is at <http://localhost:8081> (host port 8081 maps to container port 8080).

7.2 Custom Jenkins image (infra/jenkins/Dockerfile)

Step	What it does	Why
FROM jenkins/jenkins:its-jdk17	Official Jenkins long-term-support image with Java 17	Stable, supported base
USER root	Switches to root for installation	Needed to install packages
apt-get install ca-certificates curl gnupg	Tools to download and verify the Docker repository key	Secure package source
Add Docker apt repository	Registers download.docker.com with a signed key	Official Docker packages
apt-get install docker-ce-cli docker-compose-plugin	Installs only the Docker client and the compose plugin (no Docker engine)	Jenkins sends commands to the host's Docker
rm -rf /var/lib/apt/lists/*	Deletes the apt cache	Smaller image
USER jenkins	Returns to the jenkins user	Plugin install runs as jenkins
jenkins-plugin-cli --plugins ...	Pre-installs the plugins at build time	Avoids dependency errors from the setup wizard

7.3 Plugins

Installed in the image with jenkins-plugin-cli: git, workflow-aggregator (Pipeline), docker-workflow (Docker Pipeline), credentials-binding, plain-credentials, timestamp. In the setup wizard we chose "None" for suggested plugins.

7.4 Container configuration (infra/jenkins/docker-compose.yml)

- `build: .` builds the custom image above.
- `container_name: jenkins`, `restart: unless-stopped` so Jenkins comes back after a Docker or laptop restart.
- `ports: 8081 to 8080 (web UI) and 50000 (agent port).`
- `volumes`: the named volume `jenkins_home` mounted at `/var/jenkins_home` (project name jenkins gives the volume name `jenkins_jenkins_home`), so jobs, credentials and build history survive restarts and rebuilds.
- `/var/run/docker.sock` is mounted, and the container runs as `user: root`, so Jenkins can run docker and docker compose against the host's Docker.

7.5 Credentials

One credential of type **Secret file** with ID `nufindit-env`. It holds the production `.env` (DB_NAME, DB_USER, DB_PASSWORD, JWT_SECRET, TAG). The Jenkinsfile reads it with `withCredentials`, copies it to `.env` for the build, and deletes it afterwards. No secret is in Git or in the Jenkinsfile. Only `.env.example`, with placeholder values, is committed.

7.6 Job configuration

Job name `NUFindIt`, type Pipeline, definition "Pipeline script from SCM", SCM Git, repository <https://github.com/kaaaaaailll/nufindit-devops>, branch `*/main`, Script Path Jenkinsfile. The pipeline is stored in the repository (pipeline as code), not in the Jenkins UI.

7.7 Trigger configuration

The Jenkinsfile contains `triggers { pollSCM('H/2 * * * *') }`. Jenkins asks GitHub for changes about every 2 minutes (the H spreads the exact minute to avoid load spikes). If there is a new commit on main, a build starts with no manual action. A webhook would be faster, but it needs a public URL (for example ngrok), and Jenkins here runs on a laptop. Poll SCM works on any network. The Git Polling Log page in Jenkins shows each check.

7.8 Security note

Mounting the Docker socket and running as root gives Jenkins full control of the host's Docker. That is acceptable in a lab but not in production, because anyone who can change a Jenkinsfile can start privileged containers. Safer alternatives: dedicated build agents, rootless Docker, Docker-in-Docker with TLS, and Kaniko for building images without a Docker daemon.

8. Pipeline Explanation

8.1 Pipeline settings

- agent any: runs on the Jenkins controller, which has the Docker CLI.
- environment: COMPOSE_PROJECT_NAME = nufindit-devops (so every build manages the same stack) and TAG = BUILD_NUMBER (images are tagged nufindit/).
- options: timestamps (time on every log line), disableConcurrentBuilds (no two deployments at once), buildDiscarder keeping the last 15 builds.
- triggers: pollSCM every 2 minutes.

8.2 Stages

Stage	What it does	On failure
Checkout	checkout scm pulls the repository, then prints the latest commit with git log -1 --oneline	Pipeline stops, nothing changes
Test	docker build --target test for items-api and auth-api. The test stage of each Dockerfile runs the Jest tests	Build fails here. Build Images, Deploy and Smoke Test are skipped, so the running site keeps the previous version
Build Images	Loads the nufindit-env secret file as .env, then runs docker compose build. Images get the build number tag	Stops before Deploy. The old containers keep running
Deploy	docker compose up -d --no-build --remove-orphans replaces the containers with the new images. If the command fails, it waits 10 seconds and runs it once more (see 8.5). The db-data volume is kept	If the retry also fails, the stage fails and the post section prints the rollback command
Smoke Test	Up to 20 tries, 5 seconds apart. Inside the proxy container it fetches http://127.0.0.1/ and checks that the page contains "Build: ", then calls /api/items/health and /api/auth/health	After 20 tries it prints "Smoke test failed" and exits 1

Note: the smoke test uses 127.0.0.1 and not localhost. wget resolved localhost to IPv6, while nginx listens on IPv4 only, which caused "Connection refused".

8.3 Post actions

- success: prints "NUFindIt build is live at <http://127.0.0.1:8080>".
- failure: prints that the build failed and the rollback command `TAG=<previous> docker compose -p nufindit-devops up -d --no-build`.
- always: `rm -f .env`, so the secret file never stays in the workspace.

8.4 Proof of the pipeline

Build	Event	Result
#1 to #5	Setup problems (plugins, credential ID, container name conflict, IPv6)	Failed
#6	First complete run	Passed
#7	Pushed a heading change "(v2)"	Started automatically by Poll SCM. Footer showed Build: 7. Earlier items still listed
#8	Pushed a deliberately failing test (items-api/tests/break.test.js)	Failed at Test. Deploy skipped. Site stayed on Build 7
#9	Removed the failing test	Passed. Footer showed Build: 9
#10 to #18	Later pushes (README, diagram, docs and fixes)	Passed
#19	Build started while the laptop was asleep	Hung for almost 9 hours. Aborted
#20	Build triggered again	Passed

Build	Event	Result
#21	Deploy stage hit "removal of container already in progress"	Failed at Deploy
#22	After adding the retry to the Deploy stage	Passed. Rollback to build 18 and back to 22 tested
#23	Push of the technical documentation (docs only)	Started automatically by Poll SCM. Passed

8.5 Deploy retry

Build #21 failed at Deploy with "removal of container ... already in progress". Docker was still removing the old container when Compose tried to recreate it. The Deploy stage is now:

```
docker compose up -d --no-build --remove-orphans || { echo "Deploy retry after 10s"; sleep 10; docker compose up -d --no-build --remove-orphans; }
```

If the first attempt succeeds, nothing changes. If it fails, the stage waits 10 seconds and tries once more. A real failure still fails the stage on the second attempt. Build #22 passed with this change.

8.6 Build #19 (laptop sleep)

Build #19 hung for almost 9 hours because the laptop went to sleep during the run. We aborted it and ran the pipeline again (#20). Windows sleep is turned off before demos.

Note: Jenkins keeps only the last 15 builds (buildDiscarder). Builds #8 and #9 were removed from the Stages page later, but the build list screenshot above was taken while they were still shown.

9. Testing Strategy

9.1 Overview

We test at two points in the pipeline:

- Before deployment (Test stage):** automated Jest tests for both backend modules. A failure here stops the pipeline.
- After deployment (Smoke Test stage):** a live check that the new version is running and healthy.

9.2 Automated tests (10 in total)

The tests use Jest and supertest. They run inside Docker, in the test stage of each API Dockerfile, using the command `docker build --target test`. The Jenkins machine needs no Node.js.

Module	File	Test	What it checks
items-api	app.test.js	GET /health returns 200	Health endpoint answers with status ok
items-api	app.test.js	GET /items returns a list	Response is 200 and an array
items-api	app.test.js	POST /items rejects missing fields	Missing fields give HTTP 400

Module	File	Test	What it checks
items-api	app.test.js	POST /items creates an item	Valid item gives HTTP 201
items-api	validation.test.js	POST /items works without a description	Description is optional, gives HTTP 201
auth-api	app.test.js	GET /health returns 200	Health endpoint answers
auth-api	app.test.js	register rejects missing fields	Missing password gives HTTP 400
auth-api	app.test.js	register creates a user	Gives HTTP 201 and the default role student
auth-api	app.test.js	login with wrong password returns 401	Wrong password is refused
auth-api	app.test.js	login with correct password returns a token	Gives HTTP 200 and a JWT token

Both backend modules have tests, which covers the recommendation to test every backend module.

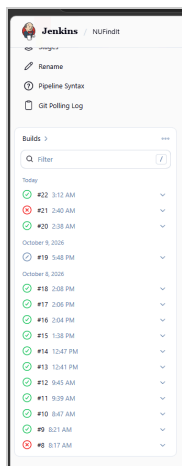
9.3 How the tests are written

Each test file builds the Express app with createApp and passes in a fake database pool (a Jest mock function). The tests therefore check the API logic (status codes, validation, password hashing and token creation) without needing a running database. They are fast and run anywhere.

9.4 How a failing test blocks deployment

The Test stage runs before Build Images, Deploy and Smoke Test. If npm test fails inside the Docker build, docker build returns an error, Jenkins marks the stage and the build as failed, and the later stages are skipped. The containers that are already running are not touched.

Proof (build #8): we pushed a deliberately failing test (items-api/tests/break.test.js). Jenkins started automatically, failed at the Test stage, and skipped Build Images, Deploy and Smoke Test. The website stayed on Build 7. After the test file was removed, build #9 passed and the site moved to Build 9.



9.5 Smoke test after deployment

After Deploy, the Smoke Test stage runs inside the proxy container. It tries up to 20 times, 5 seconds apart, and passes only when all three checks succeed:

- <http://127.0.0.1/> returns a page that contains "Build: " (the new version is live)
- <http://127.0.0.1/api/items/health> answers
- <http://127.0.0.1/api/auth/health> answers

This tests the real path through the reverse proxy to both APIs. If it never passes, the stage fails and the post section prints the rollback command.

9.6 Limitations

- The unit tests mock the database, so the real SQL and the connection to PostgreSQL are covered only indirectly, by the smoke test and the health checks.

- The frontend and the proxy have no automated tests. The smoke test covers them.
- Possible improvements: an integration test against a real test database, and a linter such as ESLint in the Test stage.

10. Rollback Procedure and Data Persistence Proof

10.1 Why rollback works

Jenkins sets TAG to the build number, so every deployment creates images named nufindit/:. Older images stay on the machine. To roll back, we start the stack again with an older TAG and the --no-build option, so Docker reuses the existing images and builds nothing.

10.2 Rollback procedure

1. Check that the old images exist: `docker images` and look for the tag, for example 18.
2. Run the rollback:

```
$env:TAG="18"; docker compose -p nufindit-devops up -d --no-build
```

3. Docker recreates proxy, frontend, items-api and auth-api from the :18 images. The db container is not touched.
4. Open <http://localhost:8080>. The footer shows "Build: 18".
5. To return to the latest version, run the same command with the newest build number, for example `$env:TAG="22"`.

Note: the rollback changes the application services only. The db image has no build tag, and the data stays in the db-data volume.

Proof: we rolled back from build 22 to build 18. All four application containers were recreated with :18 images and the db container stayed up. We then returned to build 22.

```
[+] up 5/5
✓ Container nufindit-devops-db-1 Healthy 4.6s
✓ Container nufindit-devops-auth-api-1 Started 5.0s
✓ Container nufindit-devops-frontend-1 Started 4.3s
✓ Container nufindit-devops-items-api-1 Started 4.9s
✓ Container nufindit-devops-proxy-1 Started 1.7s
```

NAME	SERVICE	IMAGE	CREATED	STATUS	PORTS	COM
nufindit-devops-auth-api-1	auth-api	nufindit/auth-api:18	21 seconds ago	Up 15 seconds (healthy)	3001/tcp	"do
cker-entrypoint.s..."	db	sha256:ab71557b737e201e24b9a47d7ba63d8d71184d57c0487b58a2d40f63b202d905	3 hours ago	Up 14 minutes (healthy)	5432/tcp	"do
nufindit-devops-frontend-1	frontend	nufindit/frontend:18	21 seconds ago	Up 16 seconds	80/tcp	"/d
cker-entrypoint.s..."	items-api	nufindit/items-api:18	21 seconds ago	Up 15 seconds (healthy)	3000/tcp	"do
nufindit-devops-proxy-1	proxy	nufindit/proxy:18	17 seconds ago	Up 15 seconds	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	"/d

10.3 Current state of the stack

All five containers are running and healthy on build 22. The only published port is 8080 on the proxy.

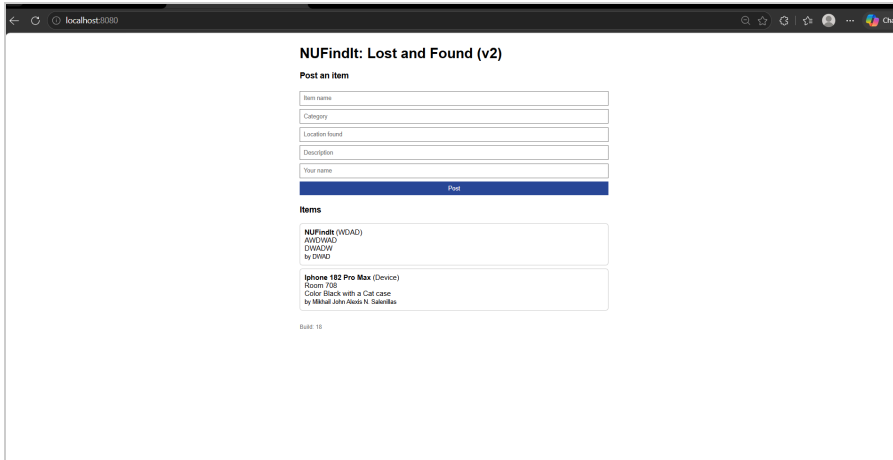
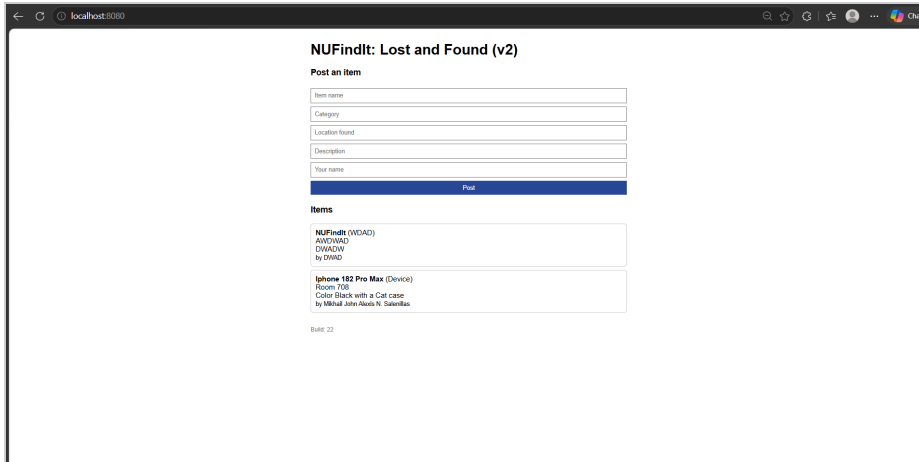
```
PS C:\Users\mikhail\nufindit-devops> docker compose -p nufindit-devops ps
```

NAME	SERVICE	IMAGE	CREATED	STATUS	PORTS	COM
nufindit-devops-auth-api-1	auth-api	nufindit/auth-api:22	About a minute ago	Up 55 seconds (healthy)	3001/tcp	"do
cker-entrypoint.s..."	db	sha256:ab71557b737e201e24b9a47d7ba63d8d71184d57c0487b58a2d40f63b202d905	36 minutes ago	Up 36 minutes (healthy)	5432/tcp	"do
nufindit-devops-frontend-1	frontend	nufindit/frontend:22	About a minute ago	Up 56 seconds	80/tcp	"/d
cker-entrypoint.s..."	items-api	nufindit/items-api:22	About a minute ago	Up 55 seconds (healthy)	3000/tcp	"do
nufindit-devops-proxy-1	proxy	nufindit/proxy:22	57 seconds ago	Up 55 seconds	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	"/d

10.4 Data persistence proof

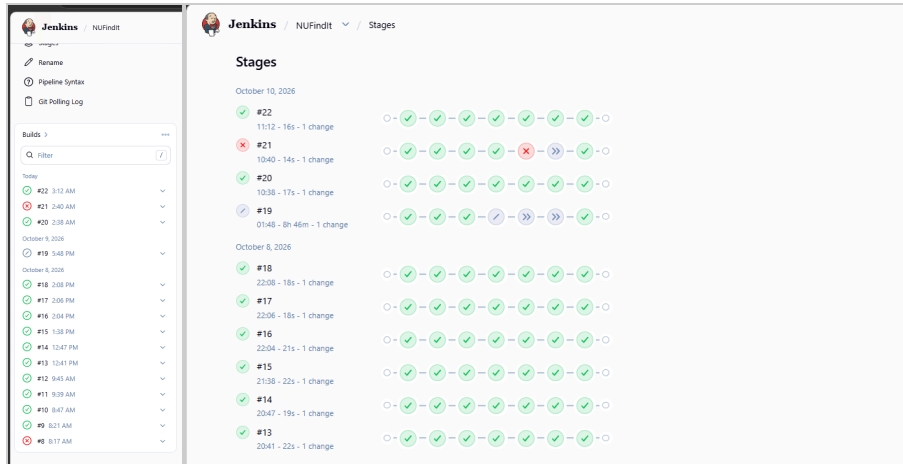
The database files live in the named volume db-data, not inside the container. We tested persistence in three ways, without ever using the -v option:

- Redeployment by Jenkins: items posted earlier were still listed after new builds were deployed (for example build #7).
- Restart: `docker compose -p nufindit-devops down` followed by `docker compose -p nufindit-devops up -d` kept all items.
- Rollback: the db container was not recreated, so its data was untouched.



10.5 Pipeline history

The Jenkins build list and Stage View show the failed build #8 (quality gate) followed by green builds.



11. Problems Encountered and How We Solved Them

#	Problem	Cause	Fix
1	Jenkins showed plugin dependency errors after "Install suggested plugins"	The setup wizard installed plugins that depend on each other in an order that failed	We baked the plugins into the Jenkins image with jenkins-plugin-cli (git, workflow-aggregator, docker-workflow, credentials-binding, plain-credentials, timestamp) and chose "None" in the setup wizard
2	The database schema could break when Jenkins deployed	The compose file bind-mounted ./db/init.sql. Under Jenkins, the path is resolved by the host's Docker, not inside the Jenkins container, so the file may not be found	We added db/Dockerfile, which copies init.sql into the image. No bind mount is needed
3	Pipeline could not find the secret file	Credential ID typos: ".env" and "nufindit-env." instead of "nufindit-env"	We corrected the ID in the Jenkins credential edit dialog so it matches the Jenkinsfile exactly
4	Deploy failed in builds #1 and #2 with a container name conflict	A leftover container from a manual run was still using the name	We removed the stack with docker compose -p nufindit-devops down (never -v, so data stays) and ran the build again
5	Smoke Test failed with "Connection refused" in builds #3 to #5	wget inside the proxy container resolved localhost to the IPv6 address, but nginx listens on IPv4 only	We changed the smoke test to use 127.0.0.1
6	auth-api and items-api went into restart loops	The code and Dockerfiles of the two APIs were mixed up (wrong ports and files in the wrong folders)	We rewrote the files in each folder and checked every file against its folder
7	Build #19 hung for almost 9 hours	The laptop went to sleep during the build	We aborted the build and ran it again (#20 passed). We now turn off Windows sleep before demos
8	Build #21 failed at Deploy with "removal of container ... already in progress"	A race condition: Docker was still removing the old container when Compose tried to recreate it	We added a one-time retry (after 10 seconds) to the Deploy stage. Build #22 passed
9	Files pasted into the VS Code editor were empty	The editor paste did not keep the content	We write files from the terminal with @'...'@ piped into Set-Content and verify them with Get-ChildItem

Lessons learned

- Keep configuration inside images (db/Dockerfile, proxy/Dockerfile) so deployments do not depend on host paths.
- A test of the real deployment path (the smoke test through the proxy) finds problems that unit tests cannot.
- Always pass the project name (-p nufindit-devops) and never use down -v, to protect the db-data volume.
- Pipeline credentials must match the Jenkinsfile exactly, character for character.

Known limitations

- The trigger is Poll SCM (up to about 2 minutes of delay) instead of a webhook, because Jenkins runs on a laptop and we did not set up a public tunnel such as ngrok.
- Jenkins uses the Docker socket and runs as root. This is acceptable for a lab only (Section 7.8).
- The db image has no build tag, so a rollback changes the application services but not the db image.
- Tags such as node:20-alpine and jenkins:its-jdk17 follow a release channel, not an exact version.

12. Individual Contributions and References

12.1 Contributions by member

Contributions are taken from the Git history of the repository.

Member	Role	Commits	Contributions
Salenillas, Mikhail John Alexis N.	DevOps / CI-CD Engineer	20 (accounts Mik1 and kaaaaaailll)	Jenkins in Docker and its plugins, Jenkinsfile, Poll SCM trigger, credentials, pipeline proofs (quality gate, rollback, persistence). Also created the initial project structure, items-api, auth-api, frontend, proxy, db, docker-compose.yml, .dockerignore files, README, architecture diagram and the technical documentation drafts
Dizon, Kizen Jared D.	Project Lead	1 (kizen19)	Created docs/project-plan.md with the task details
Aboy, Jean Ice Vincent D.	Infrastructure Engineer	1 (jeanicevincentaboy-prog)	Created docs/infrastructure-notes.md
Mallari, Ian Rose C.	Backend and Database Engineer	1 (IanMallari)	Added the items-api validation test (validation.test.js)
Flores, Nicole James B.	Frontend, QA, and Documentation Lead	3 (JamesFlores17)	Wrote testing notes (docs/testing-notes.md, later removed from the repository)

12.2 Team process

- One shared public repository on GitHub: <https://github.com/kaaaaaailll/nufindit-devops> (branch main).
- Every member has at least one commit.
- Builds started automatically from each push through Poll SCM.

12.3 References

- Docker Documentation: <https://docs.docker.com>
- Docker Compose file reference: <https://docs.docker.com/compose>
- Jenkins User Documentation (Pipeline, Credentials): <https://www.jenkins.io/doc>
- Nginx Documentation: <https://nginx.org/en/docs>
- PostgreSQL Documentation: <https://www.postgresql.org/docs>
- Node.js and Express: <https://nodejs.org> and <https://expressjs.com>
- Jest: <https://jestjs.io>
- Course guidelines: System Architecture and Integration, Final Project Guidelines (BSIT ITE 301).